iCAN 金视线杯 ·「功能模块改造与集成」实验 · 参考答案与实操指南（图文详解版）

2026 iCAN 大学生创新创业大赛「金视线杯」具身智能应用设计挑战赛 · 赛前培训资料 · 专题深化版
来源：第五轮培训录屏（2026-09-22，钱焕讲解「ROS 2 功能模块改造与集成实验」环节，全程约 49:00 → 2:06:36）逐帧复查 + 逐字稿核对。每条知识点标注**回看定位时间戳**，可对照录屏原片。
定位：这是赛前**唯一开放练习**的实验任务；比赛任务（机器人跳舞、三机联动）会比它更复杂，但"先设计通信、再落节点"的方法论一致。本册按"照着做就能调通"的顺序编排。

30 秒速览

环节	参考答案
任务全景	两台机器人（星海图 R1 · 唤醒词"你好小新" ／ 宇树 G1）**各自独立**按虚拟工程师语音命令完成：1号桌拿苹果 → 2号桌拿大象 → 3号桌按颜色放盒 → 回起点语音播报。两机之间**不通信**，都处于待命态、听到口令才动
通信骨架	每台机器人**四个节点**：主节点（业务调度）／导航／视觉／机械臂；主节点订阅 finish，发布 nav · grab · deploy，向 locate 服务发起请求
你要创建的	服务 locate（detect_type+detect_name → 点云或位姿）＋话题 nav（航点名）· grab（左右手+点云）· deploy（左右手+位姿）· finish（bool）
内置直接用	语音解析、目标检测(YOLO)、颜色检测、深度图转点云、二维点转三维点、抓取位姿估计、初始化规划组、机械臂移动、夹爪开合、导航到航点……（六大类，见 2.1 清单）
状态机	主节点 work_states **8 分支**（值 1/3/4/6/7/9/11/12）；**进分支先改状态值**防重复触发；分支序号＝画布**从左往右**顺序，节点摆好后勿拖位置
关键前置	发布**静态 TF**：手眼标定文件里 base_link↔camera_link 的位置与角度；**不填/填错→后面坐标变换全错**
省力做法	先把 **R1 全调通**跑通仿真 → "复制内容"整体复制到 G1；G1 唯一差异＝加载强化学习走路模型
调试主力	仿真中**暂停 → Debug 按钮**：看每个节点/函数的执行记录、看变量当前值；"在场景中显示"把位姿/点云画进场景核对
比赛衔接	赛前只练本任务即可（ROS2 基本通信实验不用管）；比赛指令会有拓展调整，核心思想不变；任务=三机跳舞联动；**过程分+结果分**，卡壳就走后续环节拿分

第一章 任务全貌：机器人要干什么

1.1 口令与两台机器人

虚拟工程师对着机器人喊话，例如："**你好小新**，去 1 号桌拿一个苹果，去 2 号桌拿一个大象，把苹果放到蓝色盒子里，把大象放到绿色盒子里，然后回到起点。"

- **唤醒词**：R1 的唤醒词是"你好小新"（你可以在设置唤醒词指令里自定义）；听到唤醒词后，后面的这句话被**语音解析**成若干关键词。
- **两台机器人互相独立**：R1 和 G1 都处于 STANDBY 状态，虚拟工程师先指挥 R1 完成任务，再指挥 G1 做同样的事；**两机之间没有通信**——你不要去设计跨机器人话题。
- 语音解析（提取任务信息）功能**系统已封装好**，直接调用。

1.2 一句话拆解出八个关键数据

语音解析后要提取：**4 个目的地**（1号桌、2号桌、3号桌、起点）＋**2 个抓取物**（苹果、大象）＋**2 个放置位**（蓝、绿收纳盒色号）。

`

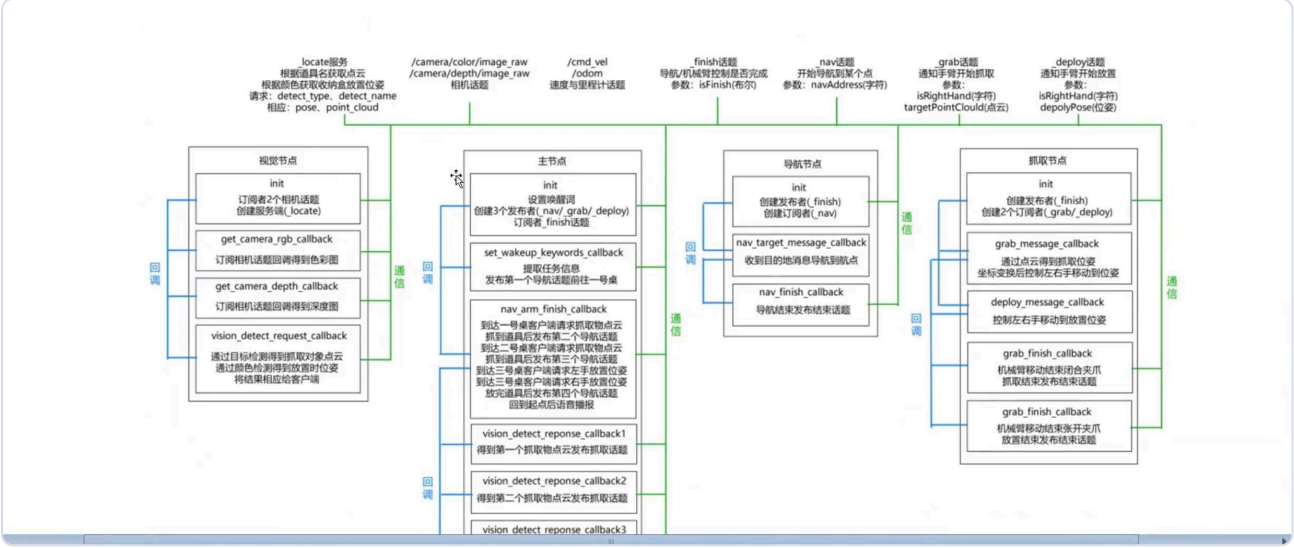
1号桌 —拿苹果—▶ 2号桌 —拿大象—▶ 3号桌 —按颜色放置—▶ 起点（语音播报"我已完成任务"）

`

1.3 单个机器人的完整流程（先想清楚，再动手）

1. 导航到 1 号桌 → 拍照 → 视觉检测苹果 → 得到**坐标变换后的点云** → 抓取位姿 → **一只手**去抓；
2. 导航到 2 号桌 → 同法检测大象 → 得点云 → **另一只手**去抓（两台桌面各用一只手；先后顺序自己定）；
3. 导航到 3 号桌 → **颜色检测**两个收纳盒中心点 → 2D 转 3D → 放置位姿 → 两只手依次放入盒子（先开的夹爪先放）；
4. 导航回起点 → **文字转语音**播报"我已完成任务" → 状态清零。

> 方法论（培训原话）：**碰到 ROS 通信问题，不要一上来就去系统里操作。先用画图的方式把"每个节点之间创建哪些话题/服务、谁发布谁订阅谁请求谁响应"画清楚，然后再进系统实现。**"很多同学一上来直接在界面里创建函数，思路不会像画图这么清晰，调试到最后连哪里错都不知道。"（回看 1:11:03、1:38:53）



▲图1｜四节点全函数设计图（参考图·干净版总览）——建议对照本册第二、三章逐节点看（会议5 回看 1:12:00）

第二章 通信设计（参考答案）

2.1 系统内置的六类指令（拿来即用，不用自己写代码）

类别	指令
通用	变量赋值、条件判断、**发布静态 TF**、TF 坐标变换
通信	创建发布者、创建订阅者、创建服务端、发布话题、客户端请求、服务端响应
语音	设置唤醒词、提取任务信息、文字转语音
导航	初始化航点、导航点、加载走路模型
视觉	目标检测、颜色检测、深度图转点云、二维点转三维点、抓取姿态估计
机械臂	初始化规划组、机械臂移动、张开夹爪、闭合夹爪

2.2 你要创建的通信资产

① 服务 locate（视觉检测服务，自定义）

项	内容
请求字段	`detect_type`（检测类型：**物体** 或 **颜色**）、`detect_name`（检测名称：如"苹果"或"蓝色"）
响应字段	`point_cloud`（点云，检测物体时返回）／`pose`（位姿，检测颜色时返回收纳盒正上方位置）
谁请求	主节点（客户端）——请求"苹果在哪/蓝色盒子上方在哪"

项	内容
谁响应	视觉节点（服务端）——按 type/name 执行检测并返回数据

> 为什么要合成一个服务？培训原话：希望"一个视觉检测服务既能检测抓取物体的位姿数据，也能检测某一个收纳盒的颜色（得到盒子正上方位姿）"——用一个服务覆盖两类视觉需求。（回看 58:04）

② 话题 finish（完成心跳，自定义）

项	内容
消息格式	布尔 `is_finish`
谁发布	导航节点（到达航点后）、机械臂节点（抓取/放置完成后）
谁订阅	**主节点**——它靠这个话题知道"上一个动作做完了"，从而推进下一步

> 导航从 A 到 B 要跑几秒、机械臂移动要几秒——**谁完成谁发 finish**，主节点收到才发下一个指令。（回看 1:00:40）

③ 话题 nav（导航目的地，自定义）

项	内容
消息格式	字符串（航点名字，如"1号桌"）
谁发布	主节点
谁订阅	导航节点

④ 话题 grab（抓取指令，自定义）

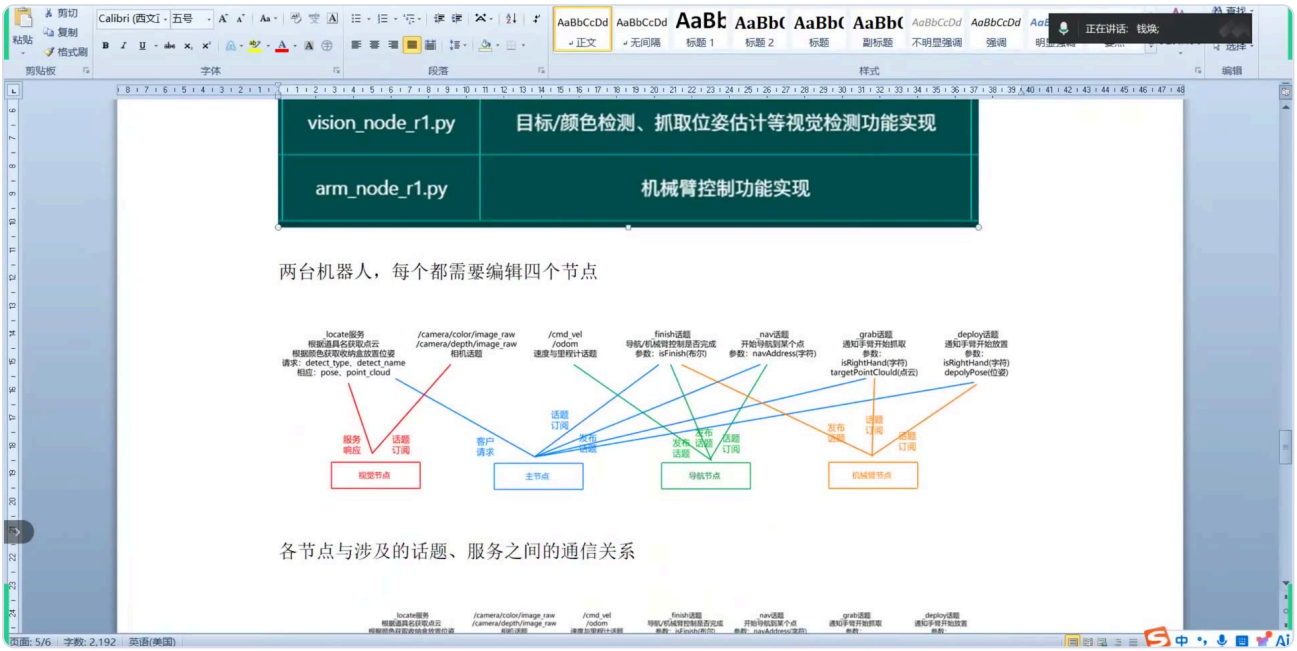
项	内容
消息格式	布尔 `hand`（用哪只手）＋ `target_point_cloud`（目标点云）
谁发布	主节点（收到 locate 响应后转发）
谁订阅	机械臂节点——做 6D 位姿估计并控制对应手臂去抓

⑤ 话题 deploy（放置指令，自定义）

项	内容
消息格式	布尔 `hand`（哪只手放）＋ `deploy_pose`（放置位姿=收纳盒正上方位置）
谁发布	主节点
谁订阅	机械臂节点

⑥ 系统内置话题（订阅即用）

话题	用途
`color_image_raw`	相机彩色图（视觉节点实时刷新）
`depth_image_raw`	相机深度图（视觉节点实时刷新）
cmd / odometry	导航的速度与里程计话题（导航内部使用）



▲ 图2｜系统内置指令与新增模块页（通用/通信/语音/导航/视觉/机械臂六类）（会议5 回看 57:00）

2.3 订阅/发布关系总览（文字版关系图）

`

虚拟工程师语音 ——唤醒词——▶ 主节点.init

主节点 ——发布 nav(1号桌)——▶ 导航节点 ——到达后发布 finish——▶ 主节点

主节点 ——请求 locate(物体/苹果)——▶ 视觉节点 ——响应 point_cloud——▶ 主节点

主节点 ——发布 grab(手=左/右, 苹果点云)——▶ 机械臂节点 ——抓取完成发布 finish——▶ 主节点

主节点 ——发布 nav(2号桌)——▶ 导航节点 ——finish——▶ 主节点

主节点 ——请求 locate(物体/大象)——▶ 视觉节点 ——响应 point_cloud——▶ 主节点

主节点 ——发布 grab(另一只手, 大象点云)——▶ 机械臂节点 ——finish——▶ 主节点

主节点 ——发布 nav(3号桌)——▶ 导航节点 ——finish——▶ 主节点

主节点 ——请求 locate(颜色/蓝)——▶ 视觉节点 ——响应 pose——▶ 主节点

主节点 ——请求 locate(颜色/绿)——▶ 视觉节点 ——响应 pose——▶ 主节点

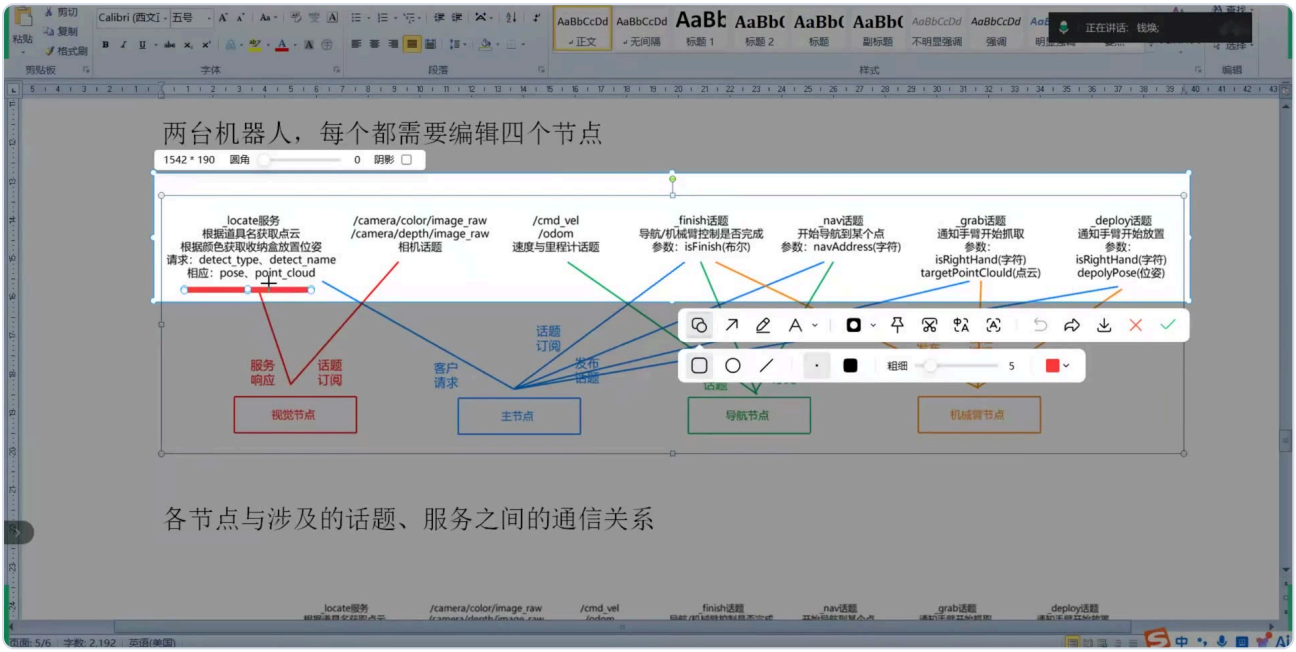
主节点 ——发布 deploy(手, 位姿)——▶ 机械臂节点 ——放完发布 finish——▶ 主节点

主节点 ——发布 nav(起点)——▶ 导航节点 ——finish——▶ 主节点

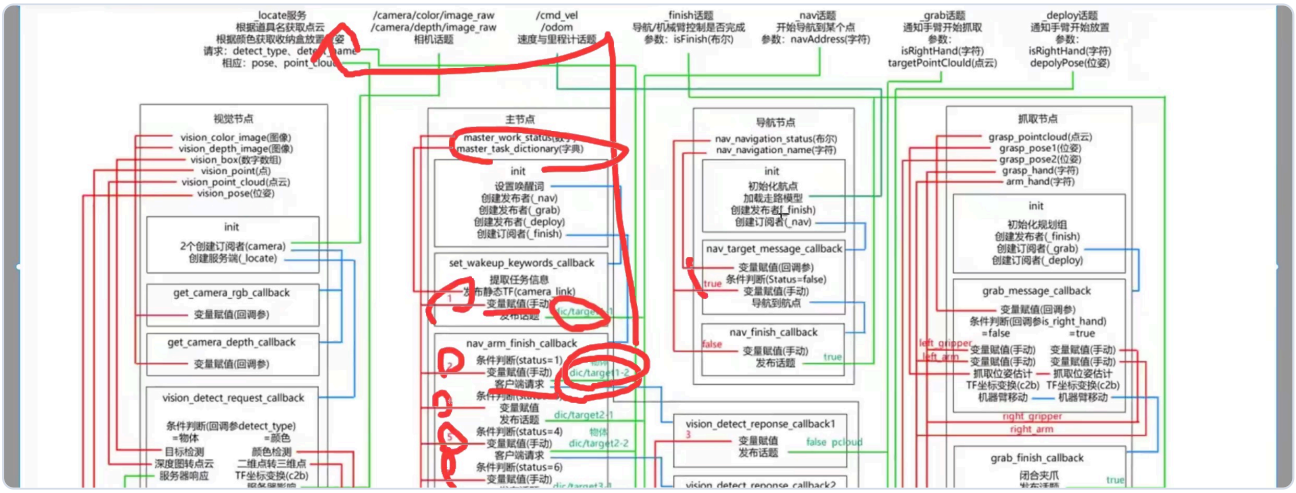
主节点 ——文字转语音："我已完成任务" ——▶ 状态清零

`

> 培训原话："导航结束干嘛？我就要请求 locate 服务去看1号桌上的苹果在哪……客户端请求服务端响应，这是双向通信，我请求完了自然有响应回来。"（回看 1:06:43）



▲ 图3｜四节点通信关系图（概念版）：两台机器人各需编辑四个节点（会议5 回看 58:20）



▲ 图4｜参考图・红圈标注讲解（合成A・含话题标签行）——讲"先画图、再动手"的方法论（会议5 回看 1:31:30 时段多帧合成）

> 注意：两次抓取**必须一左一右**——"1号桌用右手了，2号桌就只能用左手"；同理放置时"先开的夹爪先放"。（回看 1:02:20、1:03:09）

2.4 主节点视角的完整数据流（培训 1:04:28 的走读，压缩版）

1. init 提取任务信息 → 处于状态 1；

2. 发布 nav "1号桌" → 导航节点导航 → finish → 主节点进下一步；

3. 请求 locate（物体/苹果）→ 得点云；

4. 发布 grab（左/右手 + 苹果点云）→ 机械臂抓取 → finish；

5. 发布 nav "2号桌" → finish；

6. 请求 locate（物体/大象）→ 得点云 → 发布 grab（另一只手）→ finish；

7. 发布 nav "3号桌" → finish；

8. 请求 locate（颜色/蓝）→ 得 pose；请求 locate（颜色/绿）→ 得 pose；

9. 发布 deploy（第一只手 + 位姿1）→ finish → 发布 deploy（第二只手 + 位姿2）→ finish；

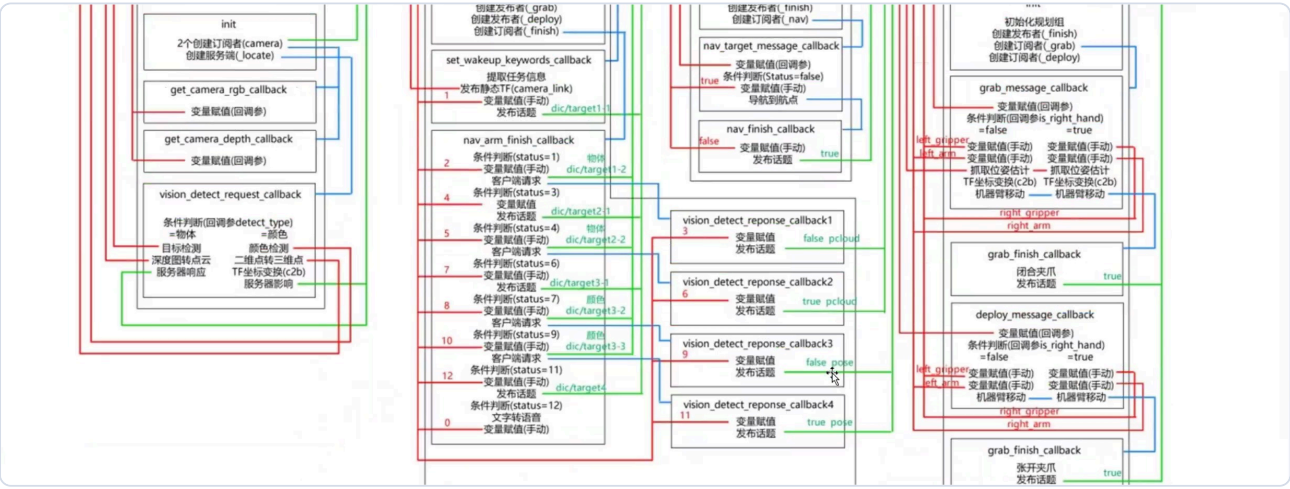
10. 发布 nav "起点" → finish → 文字转语音"我已完成任务" → work_states 清零。

第三章 四节点与变量体系

3.1 四个节点各管一摊

节点	职责	参考函数数
主节点	业务调度器：听完成信号、请求视觉、下达导航/机械臂指令、推进状态机	7 个
导航节点	接收导航目的地→执行"导航到航点"→到达后发 finish	3 个
视觉节点	实时刷新相机图像；提供 locate 服务（物体→点云／颜色→位姿）	4 个
机械臂节点	收 grab/deploy→位姿估计+TF+移动+夹爪动作→发 finish	5 个

> 培训原话："实验内部每一个机器人有四个节点……所有功能都需要在这四个节点里去实现。"（回看 56:20）



▲ 图5｜全函数设计图・红圈标注最全版（合成B）——每个节点的函数与回调关系及条件判断 1–12（会议5 回看 1:33:25 时段多帧合成）

3.2 节点变量清单（变量只在本节点内共享——"不跨节点"）

节点	变量	用途
主节点	`work_states`	当前工作状态（1/3/4/6/7/9/11/12 触发 8 个分支）
主节点	`task_dictionary`	任务字典：8 个键存目的地/抓取物/放置色
导航节点	`navigation_states`	布尔：导航是否进行中（**防冲突开关**——进行中不接受新目标）
导航节点	导航目的地名	缓存本次要去的航点名
机械臂节点	第一个抓取物点云 / 第一、第二个放置位置	缓存待抓/待放数据
机械臂节点	`grasp_hand` / `arm_hand`	左/右手规划组名、左/右臂规划组名（字符）
视觉节点	色彩图 / 深度图	实时保存最新图像（供检测用）
视觉节点	`vision_box`	检测到的矩形框（4 点数字数组）
视觉节点	`vision_point`	颜色检测的中心点（2D）
视觉节点	`vision_point_cloud`	点云（物体检测结果，发回主节点）
视觉节点	`vision_pose`	位姿（颜色检测结果，发回主节点）

> 变量用途原话："变量是用来临时存储一些数据的，然后将这些数据通过 ROS 通信方式发来发去。"（回看 1:23:45）

3.3 任务字典 8 键（target 系列）

键	含义	示例值
`target1-1`	目的地 1（1号桌航点名）	1号桌
`target1-2`	抓取物 1	苹果
`target2-1`	目的地 2	2号桌
`target2-2`	抓取物 2	大象
`target3-1`	目的地 3	3号桌
`target3-2`	放置色 1	蓝色
`target3-3`	放置色 2	绿色
`target4`	终点	起点

3.4 三色线语义（看参考图/自己画图都用这套）

- **蓝线＝回调关系**：设置唤醒词、创建订阅者、创建服务端/客户端时"绑定回调函数"的关系；
- **红线＝变量修改变量**：哪条指令改了哪个变量值（含手动赋值、回调参数赋值）；
- **绿线＝通信**：话题/服务的收发与你填的核心参数（发的是什么）。

> 培训原话逐字定义见回看 1:28:31–1:30:07。看懂参考图的方法：顺着蓝线找"被谁触发"，顺着绿线找"发出什么"，顺着红线找"谁改了状态"。

第四章 主节点实操（7 个函数）



▲ 图6｜实验环境引导页：房间场景＋「四节点编辑」「指令面板」入口（会议5 回看 1:34:20）

4.1 init（初始化）

建法（依次拖入指令）：

1. **设置唤醒词**：唤醒词填"你好小新"，回调选 `set_wake_up_keywords_callback`（下一步创建）；
2. **创建发布者 ×3**：
   - nav 发布者：消息＝`navigation_address`（字符串）
   - grab 发布者：消息＝`hand`（布尔）＋`target_point_cloud`（点云）
   - deploy 发布者：消息＝`hand`（布尔）＋`deploy_pose`（位姿）
3. **创建订阅者 ×1**：订阅 `finish`（布尔），回调选 `nav_arm_finish_callback`；
4. **变量赋值**：`work_states = 1`（把状态机置到初始值）。

> 接口格式要和话题约定**逐一对齐**，这就是"有编程思想"的体现。（回看 1:42:47）



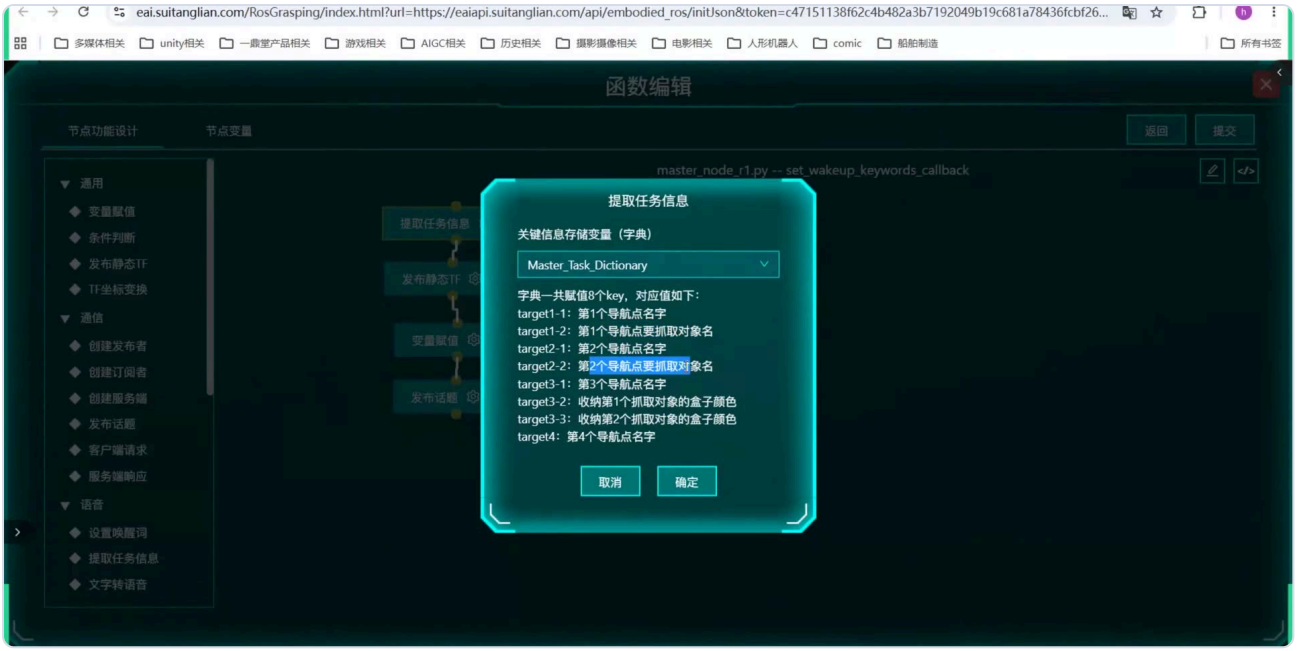
▲图7｜「ROS 节点编写」四节点函数清单（主节点 7 个函数等）（会议5 回看 1:42:00）



▲图8｜函数编辑弹窗：设置唤醒词「你好小新」＋回调函数选择（会议5 回看 1:43:10）

4.2 set_wake_up_keywords_callback（唤醒词回调）

1. **提取任务信息**：把语音里的 4 目的地＋2 抓取物＋2 放置色提取进 `task_dictionary`；
2. **发布静态 TF**（关键前置，详见 4.5）；
3. **发布 nav 话题**：值取字典 `target1-1`（第一个目的地＝1号桌）——整个任务的"第一脚"。



▲ 图9｜函数编辑弹窗：参数填写（手动赋值与选变量两种填法）（会议5 回看 1:45:00）

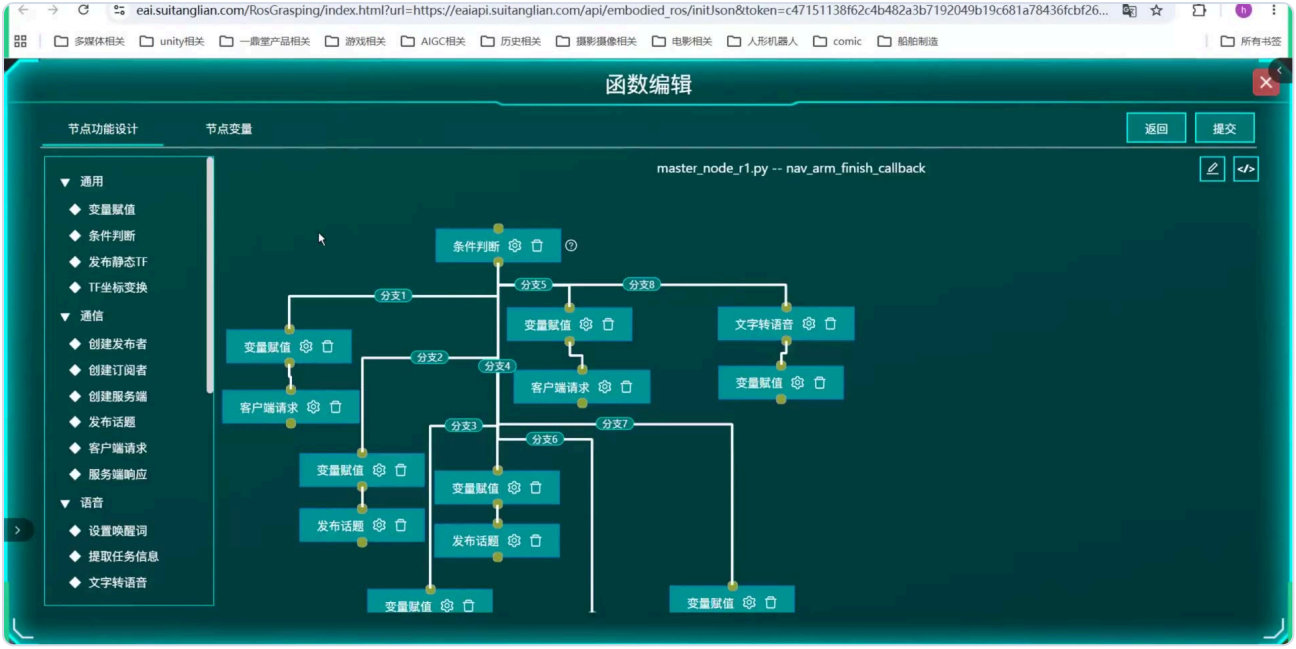
4.3 nav_arm_finish_callback（状态机主分支，8 个条件分支）

整体结构：**条件判断**，条件全部读 `work_states`，8 个分支从左往右依次摆好（**摆好后不要拖拽**——分支号按左右顺序定）。

分支	触发（work_states==）	动作（先改状态、再干活）
1	1（到达1号桌）	改 2 → 客户端请求 locate（物体, target1-2）→ 回调 `vision_detect_response_callback1`
2	3（抓取1完成）	改 4 → 发布 nav（target2-1＝2号桌）
3	4（到达2号桌）	改 5 → 客户端请求 locate（物体, target2-2）→ 回调 `...callback2`
4	6（抓取2完成）	改 7 → 发布 nav（target3-1＝3号桌）
5	7（到达3号桌）	改 8 → 客户端请求 locate（颜色, target3-2）→ 回调 `...callback3`
6	9（放置1完成）	改 10 → 客户端请求 locate（颜色, target3-3）→ 回调 `...callback4`
7	11（放置2完成）	改 12 → 发布 nav（target4＝起点）
8	12（到达起点）	文字转语音"我已完成任务" → `work_states = 0`（清零）

两条纪律（整个实验最容易被扣分的点）：

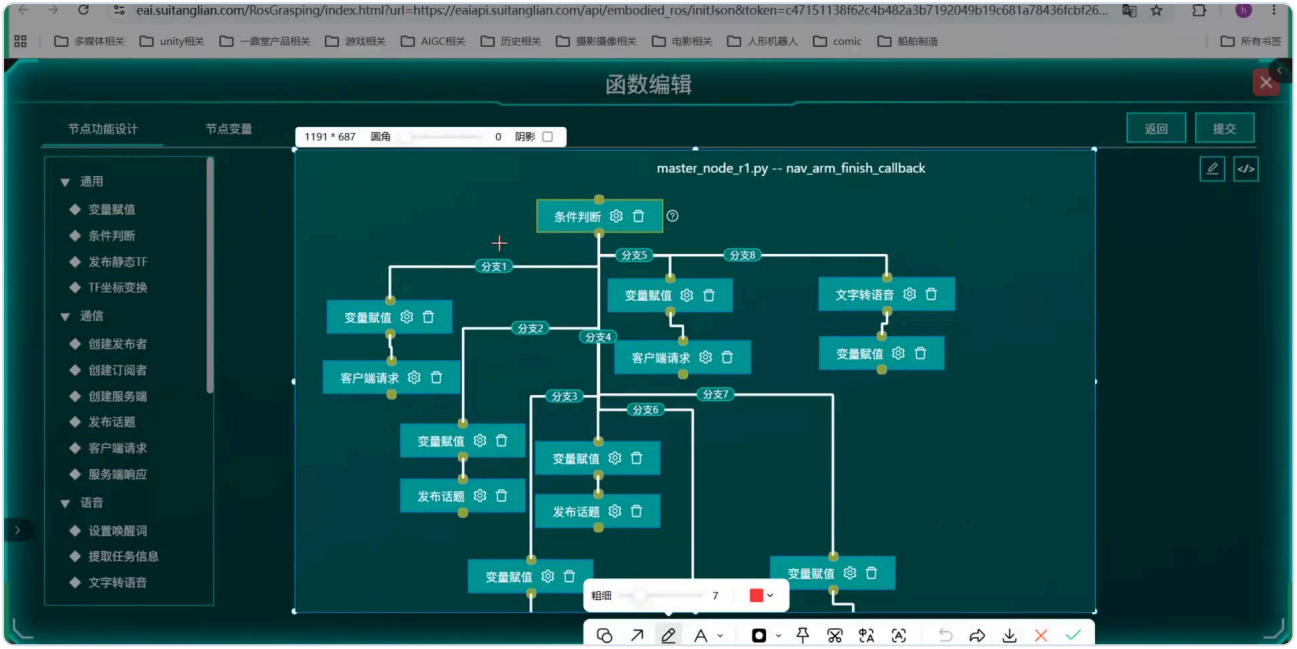
1. **进分支先改状态值**：finish 可能因多次完成信号重复触发，不先改值同一环节会执行两遍；
2. **分支顺序＝画布左右顺序**：拖拽改位会让"分支几"与内容错位。



▲图10｜状态机全画布：条件判断＋8 个分支的放射结构（会议5 回看 1:46:50）



▲图11｜条件判断分支配置弹窗：分支列表与条件（work_states==1 等）（会议5 回看 1:47:40）



▲图12｜分支内部指令链：先改变量→再客户端请求（会议5 回看 1:48:20）



▲图13｜客户端请求 locate 的参数弹窗：detect_type/detect_name＋回调选择（会议5 回看 1:50:00）

4.4 四个 vision_detect_response_callback（视觉响应回调）

统一套路：**改一个值＋发一个话题**。

回调	改值	发话题
`vision_detect_response_callback1`	work_states=3	grab（hand＋苹果点云）
`vision_detect_response_callback2`	work_states=6	grab（另一只手＋大象点云）
`vision_detect_response_callback3`	work_states=9	deploy（手1＋盒1位姿）

回调	改值	发话题
`vision_detect_response_callback4`	work_states=11	deploy（手2＋盒2位姿）

> 两次抓取必须**一左一右**；两次放置与持物手对应（先开的夹爪先放）。

4.5 发布静态 TF（坐标变换的命门）

- **为什么**：相机坐标系默认没进 TF 树——别的指令选不到相机坐标系；先发布，后面才能按相机坐标系做变换；
- **填什么**：机器人**头部相机相对基座**（base_link→camera_link）的**位置与角度**；
- **从哪抄**：外部配置文件里的**手眼标定文件**（R1 与 G1 各一份，两个机器人数值不同）；
- **怎么错**：填错→相机坐标系就是歪的→后面所有 TF 变换全出错（点云位置、抓取位姿、放置位姿全部偏）。

第五章 导航节点实操（3 个函数）

5.1 init

1. **初始化航点**（导航节点专属第一步——漏掉的话按名字导航会找不到目标）；
2. **创建发布者**：finish（布尔）；
3. **创建订阅者**：nav 话题，回调 `set_nav_goal_callback`（收目的地）。

5.2 set_nav_goal_callback（收目的地回调）

1. 回调参数＝发来的桌名 → **变量赋值**存进"导航目的地名"；
2. **条件判断**：`navigation_states == false` 才继续（否则直接丢弃——**防冲突锁**）；
3. 置 `navigation_states = true`（上锁）；
4. **导航到航点**（航点名＝刚才存的变量），设置**导航结束回调** `nav_finish_callback`。

5.3 nav_finish_callback（导航结束回调）

1. `navigation_states = false`（解锁）；
2. **发布 finish**（告诉主节点"我到了"）。

> 为什么要锁："导航是耗时动作，正在进行时又来一个新目的地，机器人行为会错乱。"——参考实现里这个开关是整个实验少见的"防御性设计"，比赛复杂度上来后这类设计只会更重要。（回看 1:54:23）



▲图14｜节点函数列表（导航/视觉区）：三函数与四函数编排（会议5 回看 1:54:00）

第六章 视觉节点实操（4 个函数）

6.1 init

1. **订阅** `color_image_raw`，回调 `get_camera_rgb_callback`；

2. **订阅** `depth_image_raw`，回调 `get_camera_depth_callback`；

3. **创建服务端**：locate（请求 detect_type + detect_name；响应 pose + point_cloud），回调 `locate_service_callback`。

6.2 两个相机回调（实时刷新）

回调里只做一件事：**变量赋值**，把最新图像存进图像变量（每帧覆盖）。服务处理时手里的图永远是最新一帧。

6.3 locate_service_callback（服务回调，两分支）

条件判断按 `detect_type` 分两路：

分支一：物体（detect_type＝物体）

1. **目标检测**：选图片（色彩图变量）＋选模型（系统内置 YOLO）；检测对象＝回调参数 `detect_name`；

2. 检测结果是**矩形框** → 存入 `vision_box`；

3. **深度图转点云**：选深度图，把矩形框**附加**到深度图上，提取框内点云 → 存 `vision_point_cloud`；

4. **服务端响应**：把点云发回主节点。

分支二：颜色（detect_type＝颜色）

1. **颜色检测**：按 `detect_name`（如"蓝色"）取画面中该色块**中心点** → 存 `vision_point`；
2. **二维点转三维点** → 得到一个位置；
3. **TF 坐标变换**：相机→基座（依赖已发布的静态 TF）；
4. **服务端响应**：把 pose 发回主节点。



▲图15｜服务端响应配置弹窗（locate 服务端参数）（会议5 回看 1:58:50）



▲图16｜服务端响应内容：点云/位姿的返回字段（会议5 回看 2:00:20）

第七章 机械臂节点实操（5 个函数）

7.1 init

1. **初始化规划组**（左手/右手/左臂/右臂等——名字要与系统配置一致）；
2. **创建发布者**：finish；
3. **创建订阅者 ×2**：grab（回调 `grab_callback`）、deploy（回调 `deploy_callback`）。

7.2 grab_callback（抓取处理）

1. **条件判断**：按 `is_right_hand` 分左右手分支；
2. 分支内：把点云送 **抓取位置估计模型** → 抓取位姿存变量；
3. **TF 坐标变换**：相机坐标系→基座坐标系；
4. **机械臂移动**（规划组名变量＋位姿变量），设**移动完成回调** `grab_finish_callback`。

7.3 grab_finish_callback（抓取完成）

1. **闭合夹爪**（选与移动手臂配套的执行器——左臂配左手夹爪）；
2. **发布 finish**。

7.4 deploy_callback（放置处理）

镜像同构：按 `is_right_hand` 分支 → 选规划组 → **机械臂移动**到 `deploy_pose` → 设**移动完成回调** `deploy_finish_callback`。

7.5 deploy_finish_callback（放置完成）

1. **张开夹爪**（物品落入收纳盒）；
2. **发布 finish**。

> 闭爪/张爪必须与移动时用的手臂**配套**，否则"手到了但爪子不动"。（回看 2:01:15）



▲图17｜抓取位置估计配置：点云→抓取位姿（模型选择）（会议5 回看 2:02:00）

第八章 单机调通 → 整体迁移到 G1

1. **先把 R1 全部调完并跑通仿真**（"你先把新海图 R1 的功能全部调完，觉得没问题了……"）；
2. 用**复制内容**把函数与指令整体复制到 G1 对应节点（无需重建）；
3. G1 唯一差异：**加载强化学习走路模型**（双足由 RL 控制走路；R1 轮式底盘直接用"导航到航点"运动）；
4. 两机各自完整跑一遍：先虚拟工程师指挥 R1，再指挥 G1——**互相独立、无通信**。



▲图18｜「ROS 节点编写」四节点界面（R1 调通后整体复制到 G1）（会议5 回看 1:35:20）

第九章 仿真运行与调试（卡点排查主力）

9.1 跑仿真怎么看

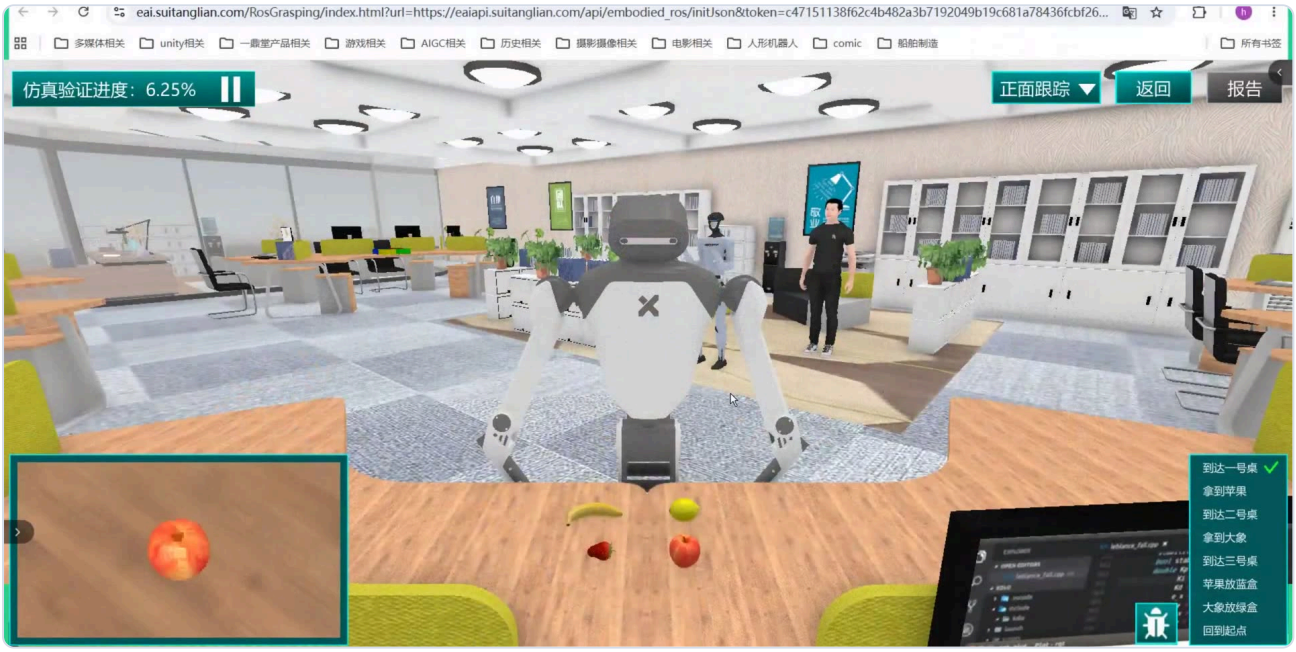
- 虚拟工程师先说话（"你好小新……"）→ 触发唤醒词 → 机器人开始动；
- **左下角**＝机器人实时看到的画面：默认俯视图，可切**正面视图**、**正面跟踪视图**（核对抓放动作的最佳视角）；
- **右下角**＝仿真进度，可**暂停**。

9.2 Debug 三件套（暂停后）

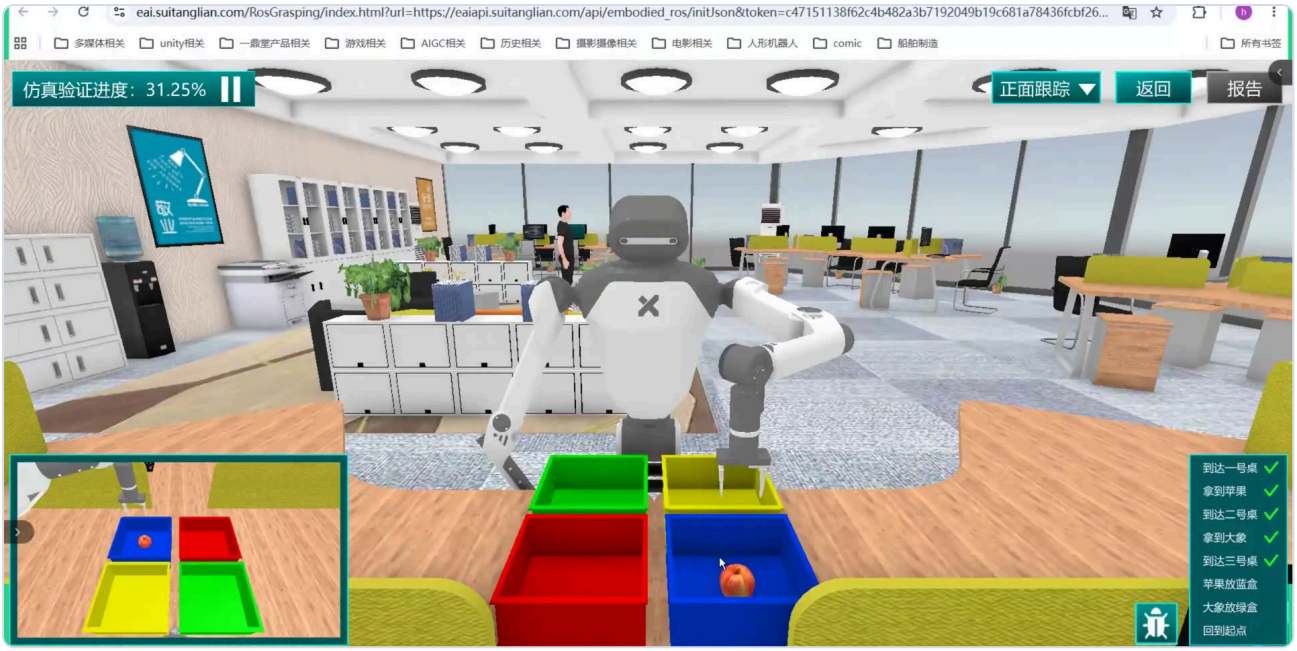
1. **执行记录**：显示仿真开始至今"哪个节点的哪个函数被执行、函数里哪些指令被执行了"——卡住时看"最后一个成功执行的函数是哪个"；

2. **变量当前值**：看任意变量的实时值（如 `grasp_pose` 抓取位姿的数值）；

3. **在场景中显示**：把变量代表的位置/点云画进 3D 场景——直接肉眼核对位姿对不对。



▲ 图19｜仿真运行：房间场景＋机器人实时视角（左下）＋进度（右下）（会议5 回看 2:03:10）



▲ 图20｜仿真近景：机器人抓放彩色收纳盒＋实时视角（会议5 回看 2:05:50）



▲ 图21｜暂停＋Debug：过程 Debug 面板（函数/指令执行记录与变量）（会议5 回看 2:04:00）

第十章 衔接正式比赛（2:06:53 起注意事项）

1. **赛前只练本任务**：培训后只需针对"功能模块改造任务"训练，**不用管 ROS2 基本通信实验**；

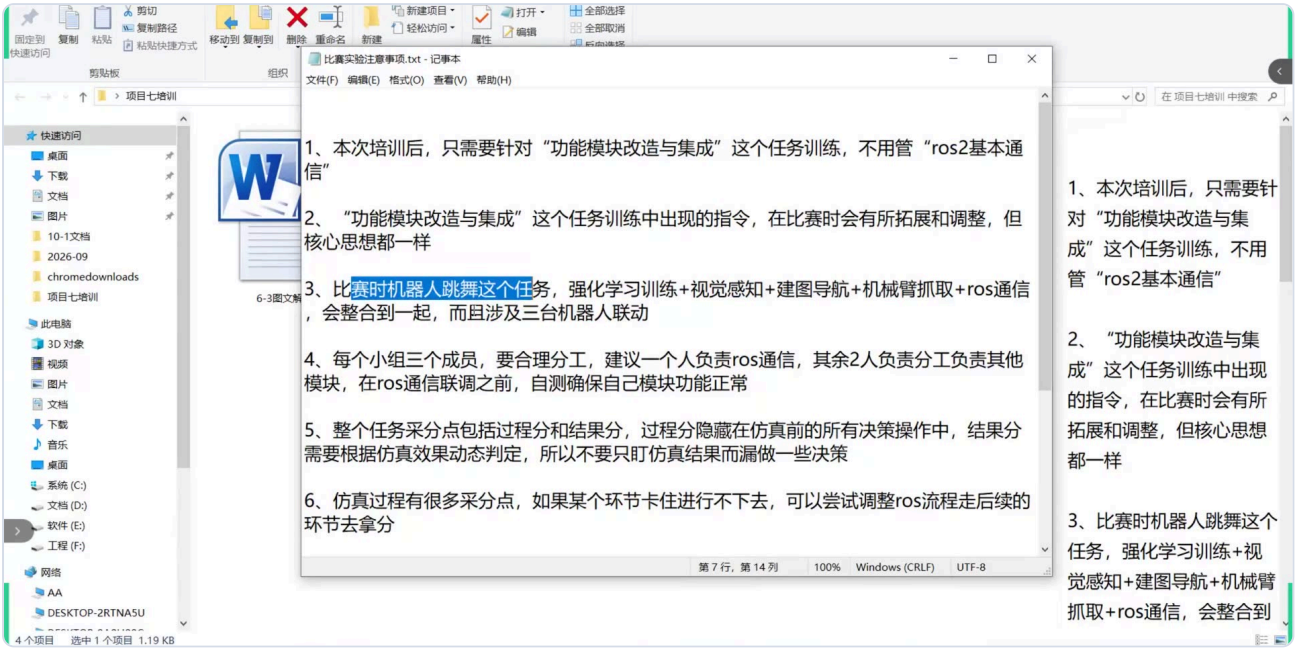
2. **比赛指令会拓展**：大多数指令你在这个实验里都会见到，但比赛会新增额外指令、现有指令也可能修改——核心思想不变："理解每个指令的作用，尝试搭建流程"；

3. **比赛任务＝机器人跳舞**：三台机器人**互相通信、联动**（不是各干各的）；且**强化学习训练、视觉感知、导航建图、机械臂抓取全部从零做起**（本实验里这些是现成的）；

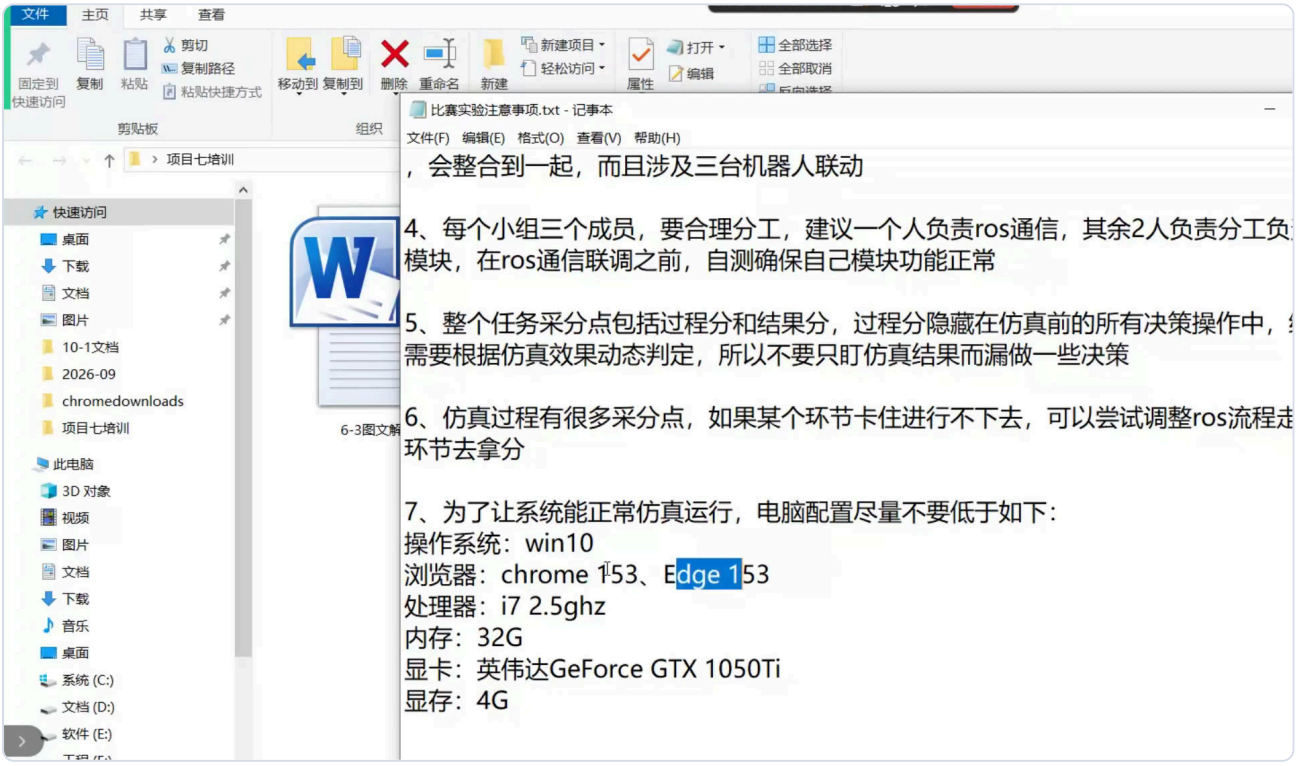
4. **分工建议（每组 3 人）**：1 人负责 ROS 通信（**自带草稿纸**，拿到题先画通信流程图再上机）；另外 2 人分头搞强化学习训练 / 视觉感知（含数据标注）/ 建图导航 / 机械臂抓取；**联调前另两人必须保证自己模块正常**，否则拖累通信联调；

5. **采分策略（重要）**：任务有**过程分＋结果分**——过程分隐含在所有决策里，结果分看仿真效果。**卡壳别死磕**：某个环节过不去就调整 ROS 流程走后续环节去拿分，不要卡在一处导致大量丢分；

6. **电脑配置**：Win10/11＋最新 Chrome/Edge；i7／2.5GHz 主频；内存 32G（16G 能跑但卡）；NVIDIA GTX 1050Ti 及以上、显存 4G+。



▲ 图22｜比赛注意事项讲解页（训练范围/跳舞任务/分工/采分）（会议5 回看 2:08:16）



▲ 图23｜系统配置要求页（i7/32G/GTX 1050Ti 及以上等）（会议5 回看 2:10:47）

附录 A｜易错点清单（照单自查）

1. 状态机分支**不先改状态值** → 同一环节重复执行（重复导航/重复请求）；

2. 分支节点**拖拽改位** → 分支编号与内容错位；

3. **静态 TF 不发布/填错** → 相机坐标系歪 → 全部 TF 变换出错（填手眼标定文件里的 base_link↔camera_link 数据）；

4. **视觉响应回调编号错配**（回调1↔物体1、回调2↔物体2、回调3↔盒1、回调4↔盒2）；

5. 两次抓取**没用不同的手** → 无法同持两件道具去放置；

6. **服务端/客户端任一侧漏设回调** → "消息发出去了但没有下一步"；

7. 导航**没有锁** → 导航进行中又收新目的地，行为错乱；

8. **跨节点读变量** → 读不到（变量不跨节点，只能靠话题/服务传）；

9. 机械臂**闭爪/张爪与手臂不配套** → 手到了爪子不动；

10. **忘了 G1 加载走路模型** → 双足不动；

11. 函数名/用途**不写清楚** → 调试时连自己都忘了这个函数干嘛的（培训原话：一定要养成写清楚的习惯）；

12. 一上来**直接进系统建函数**而不先画图 → 思路不清、调试时不知道错在哪（培训反复强调：先画图！）。

附录 B｜函数打勾表（建函数对照用）

节点	函数	用途	✓
主节点	init	唤醒词＋3 发布者＋1 订阅者＋状态置 1	☐
主节点	set_wake_up_keywords_callback	提取任务信息→字典；发布静态 TF；发第一条 nav	☐
主节点	nav_arm_finish_callback	状态机 8 分支	☐
主节点	vision_detect_response_callback1	改3＋grab 点云1	☐
主节点	vision_detect_response_callback2	改6＋grab 点云2	☐
主节点	vision_detect_response_callback3	改9＋deploy 盒1	☐
主节点	vision_detect_response_callback4	改11＋deploy 盒2	☐
导航节点	init	初始化航点＋发布者＋订阅者	☐
导航节点	set_nav_goal_callback	收目的地→判断锁→导航	☐
导航节点	nav_finish_callback	解锁＋发 finish	☐
视觉节点	init	双相机订阅＋locate 服务端	☐
视觉节点	get_camera_rgb_callback	色彩图变量刷新	☐
视觉节点	get_camera_depth_callback	深度图变量刷新	☐
视觉节点	locate_service_callback	物体/颜色两分支检测	☐
机械臂节点	init	规划组＋发布者＋双订阅者	☐
机械臂节点	grab_callback	左右手分支＋位姿估计＋tf＋移动	☐
机械臂节点	grab_finish_callback	闭爪＋finish	☐

节点	函数	用途	✓
机械臂节点	deploy_callback	左右手分支＋移动到位姿	☐
机械臂节点	deploy_finish_callback	张爪＋finish	☐

> 共 19 个函数（主 7／导航 3／视觉 4／机械臂 5），R1 调完整体复制到 G1。

> 来源说明：本册由第五轮培训录屏（2026-09-22）**逐帧复查＋逐字稿核对**编写，与《备赛重点手册・详解版》第六章同源互证；"状态值↔分支"映射按三源交叉核对结果给出（比赛前建议再用系统实际运行验证一遍）。
> 提取整理：Hermes ｜ 2026-10-17